Архитектура Quiz Bot

Данный документ содержит краткое описание основных компонентов системы. Архитектура построена на четком разграничении ответственности между контроллерами, сервисами и репозиториями, что обеспечивает масштабируемость и гибкость системы.

1. Технологический стек

- Язык программирования: Java 23.
- Фреймворк: Spring 6.2.3 (JPA, Security).
- Jetty-сервер: 11.0.20
- Telegram API: telegrambots: 6.9.7.1.
- Преобразование HTTP-запросов (Request Mapping): Jackson 2.20.0
- СУБД: PostgreSQL 17.6
- Контейнеризация: Docker 28.3.3, Docker Compose 2.39.2.
- Логирование: SLF4J 2.0.17 и Logback 1.5.18
- Генерация документов к REST API: Spring REST docs 3.0.5 Asciidoctor 3.0.5
- Тестирование: JUnit 5, Testcontainers
- Планировщик задач: Quartz 2.5.0

2. REST API Controllers

- **DemoteToUserController.java**
  Контроллер POST /admin/users/{userId}/demote для понижения пользователя с роли администратора до обычного пользователя.
- **HealthCheckController.java**
  Контроллер, обрабатывающий эндпоинт GET /healthcheck для проверки состояния приложения.
- **PromoteToAdminController.java**
  Контроллер, обрабатывающий защищенный эндпоинт
- POST /admin/users/{userId}/promote для повышения пользователя до роли администратора.
- **QuestionUploadController.java**
  Контроллер, обрабатывающий защищенный эндпоинт

- POST admin/questions/upload-csv для загрузки вопросов в систему с помощью csv-файла (form-data).
- **ResetScoreController.java**
  Контроллер, обрабатывающий защищенный эндпоинт
- POST /admin/users/{userId}/reset_score для сброса очков любого пользователя.
- **UsersController.java**
  Контроллер, обрабатывающий защищенный эндпоинт
- GET /admin/users предоставляющий функционал для получения списка всех пользователей.

3. Telegram handlers

- **DefaultHandler.java**
  Обработчик по умолчанию, который используется для обработки неизвестных или некорректных команд.
- **HealthcheckHandler.java**
  Обработчик для проверки состояния бота. Аналогичен HealthCheckController, но для Telegram.
- **HelpCommandHandler.java**
  Обработчик команды /help. Предоставляет пользователю информацию о доступных командах и их использовании.
- **StartCommandHandler.java**
  Обработчик команды /start. Используется для инициализации взаимодействия с ботом, приветствия пользователя и предоставления базовой информации, а также выбора часового пояса.
- **AddQuestionCommandHandler.java**
  Обработчик команды /add_question для добавления нового вопроса пользователем.
- **DeleteQuestionCommandHandler.java**
  Обработчик команды /delete_question <ID> для удаления вопроса. Используется для удаления вопросов из базы данных пользователем, который его и создал.
- **ShowQuestionsByTagCommandHandler.java**
  Обработчик команды /show_questions_by_tag <tag > для отображения вопросов, связанных с определённым тегом.

- **RandomByTagCommandHandler.java**
  Обработчик команды /random_by_tag <tag> для получения случайного вопроса по определённому тегу.
- **RandomCommandHandler.java**
  Обработчик команды /random для получения случайного вопроса без учёта тегов.
- **DeleteScheduleCommandHandler.java**
  Обработчик команды /delete_schedule для удаления расписания. Используется для удаления расписаний автоматической отправки случайных вопросов.
- **ScheduleCommandHandler.java**
  Обработчик команды /schedule для создания или отображения расписания.
- **ResetScoreCommandHandler.java**
  Обработчик команды /reset_score для сброса очков пользователя.
- **ScoreByTagCommandHandler.java**
  Обработчик команды /score_by_tag <tag> для отображения очков пользователя по определённому тегу.
- **ScoreCommandHandler.java**
  Обработчик команды /score для отображения общего количества очков пользователя.
- **AddTagCommandHandler.java**
  Обработчик команды /add_tag для добавления нового тега. Позволяет пользователям добавлять свои теги для вопросов.
- **DeleteTagCommandHandler.java**
  Обработчик команды /delete_tag <tag> для удаления тега. Используется для удаления ненужных тегов пользователем, который его создал.
- **ListTagsCommandHandler.java**
  Обработчик команды /list_tags для отображения списка всех доступных тегов.

4. Services

- **QuestionService.java**
  Сервис для работы с вопросами. Предоставляет методы для создания, обновления, удаления и получения вопросов.

- **ScheduleService.java**
  Сервис для управления расписаниями. Предоставляет функционал для создания, удаления и получения расписаний. Используется для настройки автоматической отправки вопросов.
- **ScoreByTagService.java**
  Сервис для работы с очками пользователей по тегам. Позволяет управлять и получать данные о результатах пользователей по определенному тегу.
- **TagService.java**
  Сервис для работы с тегами. Предоставляет методы для добавления, удаления и получения тегов. Используется для классификации вопросов.
- **UserService.java**
  Сервис для работы с пользователями. Управляет данными пользователей, включая их роли и счёт.
- **BaseQuizService.java**
  Базовый сервис для викторин. Содержит логику обработки результатов ответов на вопросы.
- **QuizByTagService.java**
  Сервис для получения случайного вопроса из базы данных.
- **QuizService.java**
  Сервис для получения случайного вопроса по тегу из базы данных.

5. Repositories

Репозитории предоставляют доступ к данным в базе, используя решения Spring Data и JPA

- **QuestionRepository.java**
  Репозиторий для работы с сущностью Question. Предоставляет методы для сохранения, обновления, удаления и поиска вопросов.
- **ScheduleRepository.java**
  Репозиторий для работы с сущностью Schedule. Управляет доступом к данным расписаний, включая их создание, обновление, удаление и поиск. Содержит методы для получения расписаний, связанных с конкретным пользователем.
- **ScoreByTagRepository.java**
  Репозиторий для работы с сущностью ScoreByTag. Предоставляет

доступ к данным о результатах пользователей, сгруппированных по тегам. Включает методы получения очков по тегам.

- **TagRepository.java**
  Репозиторий для работы с сущностью Tag. Управляет доступом к данным тегов, включая их создание, удаление и поиск. Содержит методы для получения списка всех тегов или поиска тегов по названию.
- **UserQuestionRepository.java**
  Репозиторий для работы с сущностью UserQuestion. Используется для хранения вопросов, на которые уже ответил пользователь.
- **UserRepository.java**
  Репозиторий для работы с сущностью User. Управляет доступом к данным пользователей, включая их создание, обновление, удаление и поиск. Содержит методы для поиска пользователей по Telegram ID и другим параметрам.

6. Интеграционные компоненты

Telegram API – получает сообщения от пользователей, преобразует их в объекты, понятные внутреннним компонентам приложения, а также преобразует ответы системы в сообщения для пользователей.

7. Развертывание приложения.

Для развёртывания приложения использовался контейнерный подход: приложение собирается в **fat-jar** при помощи Gradle и упаковывается в Docker multi-stage образ. Для локального развёртывания и оркестрации сервисов используется **docker compose**. В качестве СУБД используется PostgreSQL в отдельном контейнере.

7.1. Сборка Docker-образа.

В проекте используется multi-stage Dockerfile: на первой стадии — build — запускается JDK и Gradle, собирающий **fat-jar** (shadowJar). На второй стадии — runtime — используется JRE, в конечный образ копируется только готовый jar и вспомогательный entrypoint.sh. Такой подход даёт компактный финальный образ без инструментов сборки.

Пример Dockerfile:

```
FROM eclipse-temurin:23-jdk AS build
WORKDIR /workspace
COPY . /workspace
RUN ./gradlew clean shadowJar -x test

FROM eclipse-temurin:23-jre
WORKDIR /app
COPY --from=build /workspace/build/libs/*.jar /app/app.jar
COPY .env /app/.env

CMD ["java", "-jar", "/app/app.jar"]
```

7.2. Оркестрация (docker compose)

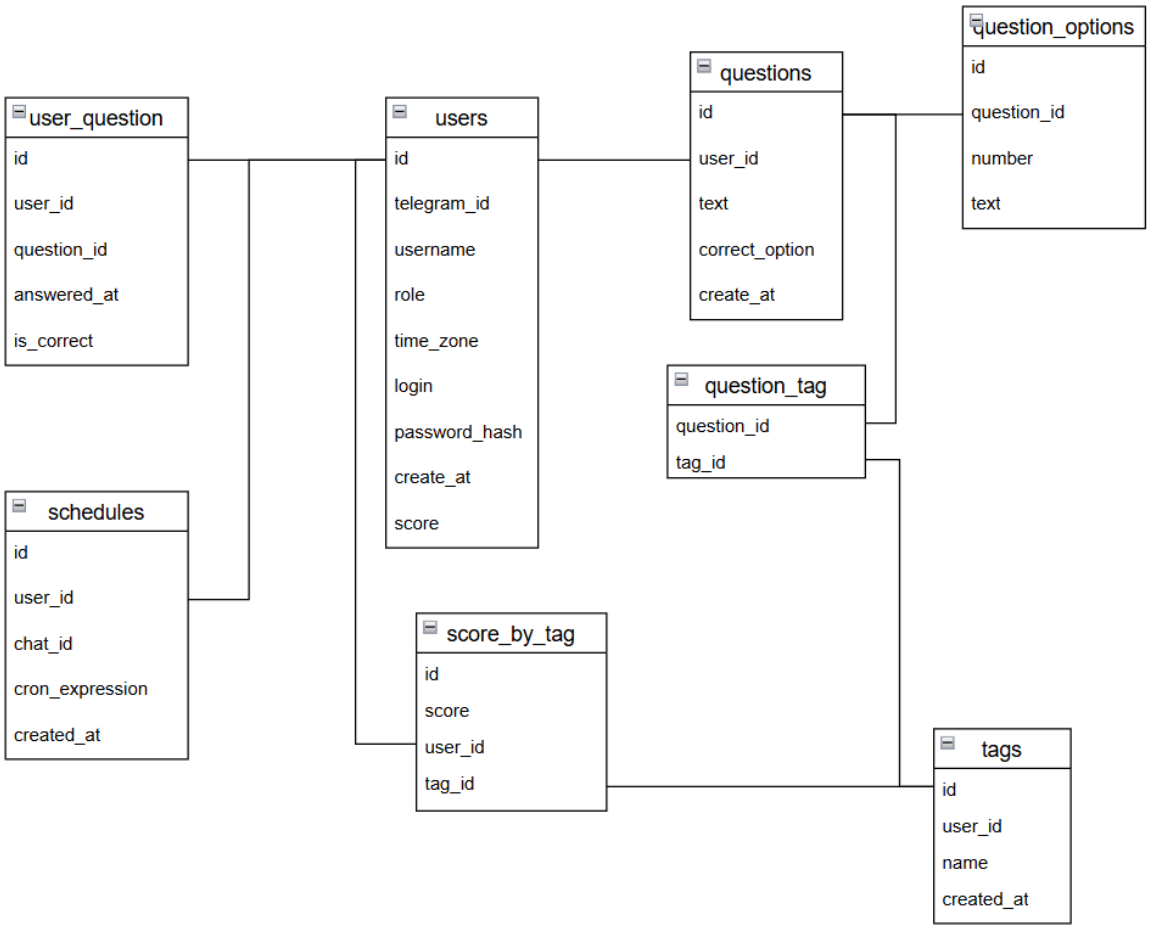
Для локальной оркестрации и тестового развёртывания используется docker compose. В docker-compose.yml описаны два сервиса: postgres и app. Основные аспекты оркестрации:

Для локальной оркестрации и тестового развёртывания используется docker compose. В docker-compose.yml описаны два сервиса: postgres и app. Основные аспекты оркестрации:

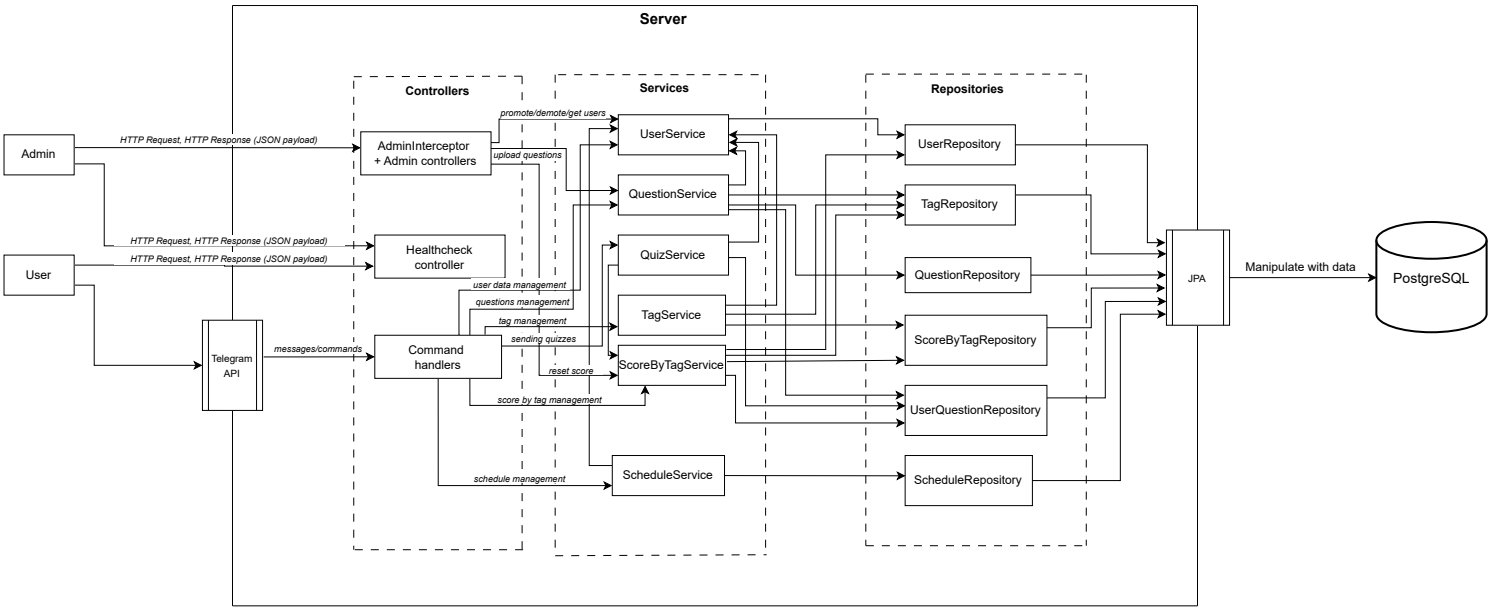
- postgres запускается с декларацией тома pgdata для персистентного хранения.
- app собирается из текущего репозитория (build: .) и объявлен как зависящий от postgres. depends_on гарантирует порядок старта, но не готовность — поэтому в postgres добавлен healthcheck (pg_isready).
- Оба сервиса подключены к общей сети quiznet.
- Том pgdata гарантирует сохранность данных при перезапуске или обновлении контейнера.
- app пробрасывает порт 8080:8080 для доступа извне.

- restart: unless-stopped — простая политика для single-host окружения.

8. ER-диаграмма модели данных



9. Диаграмма приложения


Server
Controllers
Services
Repositories
Admin
User
HTTP Request, HTTP Response (JSON payload)
HTTP Request, HTTP Response (JSON payload)
HTTP Request, HTTP Response (JSON payload)
AdminInterceptor
+ Admin controllers
Healthcheck
controller
Telegram
API
messages/commands
Command
handlers
promote/demote/get users
upload questions
user data management
questions management
tag management
sending quizzes
reset score
score by tag management
schedule management
UserService
QuestionService
QuizService
TagService
ScoreByTagService
ScheduleService
UserRepository
TagRepository
QuestionRepository
ScoreByTagRepository
UserQuestionRepository
ScheduleRepository
JPA
Manipulate with data
PostgreSQL